

Vérification formelle légère d'une blockchain Akka/Scala

Réseau de Pétri, invariants et propriétés LTL

Allauguillemette Tom Vinkovic Matias Chekhab Elias Ouguerd Ayman
Saghir Louaye

Mai 2026

Table des matières

1	Résumé	2
2	Présentation du système réel	2
2.1	Architecture Akka Typed	2
2.2	Flux nominal d'un transfert	3
2.3	Flux d'échec important : <code>DB.Failed</code>	4
3	Modèle formel par réseau de Pétri	4
3.1	Définition simplifiée	4
3.2	Niveau d'abstraction	4
3.3	Places principales	5
3.4	Transitions principales	5
4	Scénarios de vérification	6
5	Vérification des propriétés structurelles	7
5.1	Définition des propriétés structurelles	7
5.2	Absence de deadlock interne	7
5.3	Non-suppression avant confirmation	8
5.4	Conservation de la mempool après <code>DB.Failed</code>	8
5.5	GetTxs est non destructif	8
5.6	Exclusion entre minage et attente DB	9
5.7	Validité de l'entrée en mempool	9
5.8	Refus local en cas de fonds insuffisants	9
6	Invariants	9
6.1	Invariants structurels de places	9
6.2	Invariants métier	11
6.3	Point d'attention sur le solde	11

7 Propriétés LTL	11
7.1 Rappel simple sur LTL	11
7.2 Propositions atomiques	12
7.3 Formules vérifiées	12
7.4 Résultat sur les traces de scénarios	13
7.5 Résultat sur l'espace d'états borné	13
7.6 Pourquoi il faut des hypothèses de justice	13
8 Comparaison entre comportement réel et modèle formel	14
8.1 Correspondance étape par étape	14
8.2 Ce que le modèle reproduit fidèlement	15
8.3 Différences entre le modèle et le code réel	15
8.4 Écart métier principal : échec DB et retry implicite	16
9 Résultats synthétiques	16
10 Limites de la vérification	17
11 Conclusion	17

1 Résumé

Ce rapport présente une vérification formelle légère d'une simulation de blockchain écrite en Scala avec Akka Typed. Le système réel est un backend distribué par acteurs : un registre gère les wallets, les wallets créent et signent les transactions, la mempool stocke les transactions valides, un validateur mine périodiquement un bloc, puis une base de données persiste le bloc dans un ledger.

L'objectif du modèle formel n'est pas de recalculer tous les détails numériques de la blockchain. Il sert plutôt à vérifier le **flux de contrôle critique** : une transaction ne doit pas être supprimée avant confirmation, un bloc miné doit recevoir une réponse de la base de données, un échec de persistance ne doit pas nettoyer la mempool, et le validateur ne doit pas lancer deux persistances concurrentes du même cycle.

La méthode utilisée combine :

- un **réseau de Pétri** exécutable dans `petri-blockchain-visualizer/index.html` ;
- des **scénarios témoins** correspondant aux chemins importants du code ;
- des **invariants structurels** sur les places et les transitions ;
- des propriétés temporelles en **LTL** évaluées sur les traces et, sous hypothèses bornées, sur l'espace d'états exploré ;
- une comparaison entre le comportement réel Akka et le comportement abstrait du modèle.

La conclusion principale est que le modèle capture correctement les propriétés de sûreté essentielles : `RemoveTx` ne se produit qu'après `DB.Success`, `DB.Failed` ne nettoie pas la mempool, les transactions invalides sont rejetées, et l'état `waitingForDbState` empêche le validateur de lancer un second minage. La principale limite concerne la vivacité globale : elle dépend d'hypothèses de justice et de bornes, car un réseau de Pétri purement non déterministe peut toujours construire des traces artificielles qui retardent indéfiniment une réponse.

2 Présentation du système réel

2.1 Architecture Akka Typed

Le projet est structuré autour de plusieurs acteurs Akka Typed. Chaque acteur possède un état local et communique uniquement par messages typés. Cette architecture se prête bien à une modélisation par réseau de Pétri, car les messages deviennent naturellement des transitions et les états internes deviennent des places.

Acteur / module	Rôle dans le code	Éléments modélisés dans le réseau de Pétri
RegistryActor	Maintient une table <code>name -> ActorRef[Wallet.Command]</code> . Crée des wallets et résout un wallet par son nom.	Places <code>P_REGISTRY_READY</code> , <code>P_REGISTRY_HAS_WALLET</code> , <code>P_REGISTRY_NO_WALLET</code> . Transitions <code>CreateWallet</code> et <code>GetWalletRef</code> .
WalletActor	Gère les clés RSA simplifiées, le solde local, le nonce, la création et la signature des transactions.	Places <code>P_WALLET_IDLE</code> , <code>P_WALLET_BUSY</code> , <code>P_WALLET_BALANCE_REQUEST</code> , <code>P_UNSIGNED_TX</code> , <code>P_SIGNED_TX_SENT</code> , <code>P_TX_REJECTED_LOCAL</code> .
MempoolActor	Vérifie les signatures, stocke les transactions en attente, trie par fees décroissants, fournit les deux meilleures transactions au validateur.	Places <code>P_MEMPOOL_EMPTY</code> , <code>P_MEMPOOL_TX_PENDING</code> , <code>P_MEMPOOL_ADD_REQ</code> , <code>P_MEMPOOL_BATCH_SENT</code> , <code>P_MEMPOOL_CLEANUP_REQ</code> .

Acteur / module	Rôle dans le code	Éléments modélisés dans le réseau de Pétri
ValidatorActor	Interroge la mempool toutes les 5 secondes, demande le dernier bloc, mine un nouveau bloc, attend la réponse DB, confirme ou rejette les transactions.	Places P_VALIDATOR_IDLE, P_VALIDATOR_POLLING, P_VALIDATOR_BATCH, P_MINING, P_BLOCK_MINED, P_VALIDATOR_WAITING_DB.
DBActor	Persiste les blocs dans ledger.txt, maintient lastHash, currentId et une vue mémoire des blocs. Calcule aussi les soldes depuis le ledger.	Places P_DB_LEDGER, P_DB_BALANCE_REQ, P_DB_LASTBLOCK_REQ, P_DB_SAVE_IN_PROGRESS, P_BLOCK_PERSISTED, P_DB_SAVE_FAILED.

2.2 Flux nominal d'un transfert

Le chemin nominal d'un transfert confirmé est le suivant :

1. L'API HTTP reçoit une requête POST /api/wallets/:name/transfer.
2. La route demande au RegistryActor de résoudre le wallet avec GetWalletRef.
3. Si le wallet existe, la route envoie Wallet.CreateTx au WalletActor.
4. Le wallet demande son solde à la DB, puis construit une UnsignedTransaction si les fonds sont suffisants.
5. Le wallet calcule le hash, signe la transaction, puis envoie Mempool.AddTx.
6. La mempool vérifie la signature ; si elle est valide, elle insère la transaction dans une liste triée par fees.
7. Toutes les 5 secondes, le ValidatorActor reçoit StartMining, demande GetTxS, puis mine un bloc si la mempool n'est pas vide.
8. Le validateur demande DB.GetLastBlock, calcule le proof of work, puis envoie DB.SaveBlock.
9. En cas de DB.Success, le validateur répond true aux wallets concernés et demande Mempool.RemoveTxS.
10. La mempool supprime les transactions confirmées. Le système revient dans un état stable.

En notation de transitions du modèle, le scénario principal est :

```

T_REGISTRY_GET_WALLET_FOUND
T_WALLET_CREATE_TX_START
T_WALLET_GET_BALANCE_REQUEST
T_DB_BALANCE_RESPONSE
T_WALLET_FUNDS_OK_BUILD_UNSIGNED
T_WALLET_SIGN_SEND_MEMPOOL
T_MEMPOOL_ADD_TX
T_MEMPOOL_VERIFY_OK_ADD_EMPTY
T_VALIDATOR_START_MINING
T_MEMPOOL_GET_TXS_NONEMPTY
T_DB_GET_LAST_BLOCK_REQUEST
T_DB_GET_LAST_BLOCK_RESPONSE
T_MINER_START
T_MINER_PROOF_OF_WORK
T_DB_SAVE_BLOCK_REQUEST
T_DB_SAVE_SUCCESS
T_VALIDATOR_CONFIRM_SAVED
T_MEMPOOL_REMOVE_CONFIRMED_ALL

```

2.3 Flux d'échec important : `DB.Failed`

Le comportement en cas d'échec DB est particulièrement important, car il révèle un choix métier du code réel. Dans `ValidatorActor`, si `DB.SaveBlock` échoue, le validateur exécute `SaveFailed` :

- il répond `false` aux `replyTo` des transactions sélectionnées ;
- il repasse en état normal ;
- il **ne déclenche pas** `Mempool.RemoveTx`.

La conséquence est que les transactions restent dans la mempool. Formellement, ce n'est pas un deadlock : un prochain tick du validateur peut reprendre les transactions. En revanche, côté API, les utilisateurs ont déjà reçu une réponse négative. Ce point doit être assumé comme une politique de retry implicite, ou corrigé si l'intention est de rejeter définitivement la transaction après un échec DB.

3 Modèle formel par réseau de Pétri

3.1 Définition simplifiée

Un réseau de Pétri peut être vu comme un graphe biparti composé de :

- **places** : elles représentent des états du système ;
- **transitions** : elles représentent des événements ou des messages ;
- **jetons** : ils indiquent quels états sont actuellement vrais ;
- **arcs** : ils indiquent quelles places sont consommées et produites par chaque transition.

Un marquage M associe à chaque place un nombre de jetons. Une transition t est activable si toutes ses places d'entrée possèdent au moins un jeton. Quand t est tirée, elle consomme un jeton dans chaque place d'entrée et produit un jeton dans chaque place de sortie.

Dans ce projet, une transition du réseau correspond généralement à un message ou à une étape importante du code Akka. Par exemple :

- `T_MEMPOOL_ADD_TX` représente la réception de `Mempool.AddTx` ;
- `T_DB_SAVE_BLOCK_REQUEST` représente l'envoi de `DB.SaveBlock` ;
- `T_DB_SAVE_SUCCESS` représente la réponse positive `DB.Success` ;
- `T_VALIDATOR_SAVE_FAILED` représente le traitement de `DB.Failed` par le validateur.

3.2 Niveau d'abstraction

Le modèle est volontairement une abstraction du code réel. Il représente le contrôle, mais pas tous les calculs métier. Sont abstraits :

- les montants exacts ;
- les fees exacts ;
- le nonce exact ;
- les timestamps ;
- la valeur réelle des hashes ;
- la cryptographie RSA réelle ;
- le nombre exact de transactions dans la mempool.

À la place, le réseau distingue les états logiques importants : *mempool vide*, *mempool non vide*, *signature valide*, *signature invalide*, *fonds suffisants*, *fonds insuffisants*, *DB success*, *DB failed*. Ces choix rendent le modèle plus simple à comprendre tout en gardant les propriétés importantes vérifiables.

3.3 Places principales

Place	Interprétation
P_REGISTRY_READY	Le registre est disponible pour créer ou résoudre des wallets.
P_REGISTRY_HAS_WALLET	Le registre contient le wallet demandé.
P_REGISTRY_NO_WALLET	Le wallet demandé est absent.
P_WALLET_IDLE	Le wallet n'est pas en train de traiter une transaction.
P_WALLET_BUSY	Le wallet traite une transaction ; cela correspond à <code>txInProgress = true</code> .
P_WALLET_BALANCE_REQUEST	Le wallet attend le résultat de <code>DB.GetBalanceAtDate</code> .
P_UNSIGNED_TX	La transaction non signée a été construite.
P_SIGNED_TX_SENT	La transaction signée a été envoyée à la mempool.
P_MEMPOOL_EMPTY	La mempool ne contient aucune transaction abstraite.
P_MEMPOOL_TX_PENDING	La mempool contient au moins une transaction valide.
P_MEMPOOL_BATCH_SENT	La mempool a envoyé un lot de transactions au validateur sans les supprimer.
P_DB_LASTBLOCK_REQ	Le validateur attend <code>DB.LastBlockInfo</code> .
P_MINING	Le proof of work est en cours.
P_BLOCK_MINED	Le bloc est miné mais pas encore persisté.
P_DB_SAVE_IN_PROGRESS	Une écriture <code>DB.SaveBlock</code> est en attente.
P_VALIDATOR_WAITING_DB	Le validateur attend la réponse de la DB. Les nouveaux ticks sont ignorés.
P_BLOCK_PERSISTED	La DB a répondu <code>Success</code> .
P_DB_SAVE_FAILED	La DB a répondu <code>Failed</code> .
P_SIGNATURE_INVALID	Une transaction a été rejetée par la mempool à cause d'une signature invalide.
P_VALIDATOR_REJECT_TXS	Le validateur doit répondre négativement aux transactions sélectionnées après un échec DB.
P_MEMPOOL_CLEANUP_REQ	Une demande de suppression des transactions confirmées a été envoyée.

3.4 Transitions principales

Transition	Code représenté	Effet formel
T_REGISTRY_GET_WALLET_FOUND	<code>Registry.GetWalletRef</code> réussi	Produit une référence de wallet.
T_WALLET_CREATE_TX_START	<code>Wallet.CreateTx</code>	Passe le wallet en traitement et déclenche une demande de solde.
T_DB_BALANCE_RESPONSE	<code>DB.GetBalanceAtDate</code>	Retourne le solde calculé depuis le ledger.
T_WALLET_FUNDS_OK_BUILD_UNSIGNED	Test <code>currentBalance >= amount + fee</code>	Construit une transaction non signée.
T_WALLET_FUNDS_INSUFFICIENT	Fonds insuffisants	Rejette localement la transaction.
T_WALLET_SIGN_SEND_MEMPOOL	<code>Crypto.sign</code> puis <code>Mempool.AddTx</code>	Produit une transaction signée envoyée à la mempool.
T_MEMPOOL_VERIFY_OK_ADD_EMPTY	Signature valide, mempool vide	Ajoute la transaction et rend la mempool non vide.

Transition	Code représenté	Effet formel
T_MEMPOOL_VERIFY_OK_ADD_NONEMPTY	Signature valide, mempool déjà non vide	Ajoute et trie la transaction, la mempool reste non vide.
T_MEMPOOL_VERIFY_INVALID	Signature invalide	Rejette la transaction sans modifier la mempool.
T_VALIDATOR_START_MINING	Tick StartMining	Le validateur interroge la mempool.
T_MEMPOOL_GET_TXS_NONEMPTY	GetTxs avec transactions	Envoie un batch sans retirer les transactions.
T_MEMPOOL_GET_TXS_EMPTY	GetTxs vide	Le validateur revient idle, aucun bloc n'est miné.
T_DB_GET_LAST_BLOCK_REQUEST	DB.GetLastBlock	Demande le dernier hash et l'id courant.
T_MINER_PROOF_OF_WORK	Miner.mine	Produit un bloc miné abstrait.
T_DB_SAVE_BLOCK_REQUEST	DB.SaveBlock	Démarre la persistance et place le validateur en attente DB.
T_DB_SAVE_SUCCESS	Réponse DB.Success	Le bloc est persisté ; le validateur peut confirmer.
T_DB_SAVE_FAILED	Réponse DB.Failed	La DB garde son ancien état ; le validateur peut rejeter.
T_VALIDATOR_CONFIRM_SAVED	ConfirmSaved	Répond true et demande RemoveTxs.
T_VALIDATOR_SAVE_FAILED	SaveFailed	Répond false sans nettoyer la mempool.
T_MEMPOOL_REMOVE_CONFIRMED_ALL	RemoveTxs	Supprime les transactions confirmées, mempool vide.
T_MEMPOOL_REMOVE_CONFIRMED_PARTIAL	RemoveTxs partiel	Supprime les confirmées mais conserve un reste.
T_VALIDATOR_TICK_IGNORED	waitingForDbState: case _ => Behaviors.same	Ignore un tick pendant l'attente DB.

4 Scénarios de vérification

Le visualiseur contient plusieurs scénarios. Ils servent de traces témoins : chaque scénario rejoue une séquence représentative du code réel.

Scénario	Objectif vérifié
transfer_success	Chemin nominal complet : wallet trouvé, transaction créée, ajoutée à la mempool, minée, persistée, puis supprimée.
transfer_success_mempool_nonempty	Même chemin nominal, mais avec une mempool déjà non vide. Vérifie que l'ajout et le retrait partiel sont possibles.
db_failure_retry_possible	Échec de DB.SaveBlock. Vérifie que la transaction est rejetée côté reply mais non supprimée de la mempool.
insufficient_funds	Refus local du wallet avant signature et avant entrée en mempool.

Scénario	Objectif vérifié
wallet_not_found	Refus API lorsque le registre ne trouve pas le wallet.
create_wallet_success	Création d'un wallet absent.
create_wallet_exists	Refus de création si le wallet existe déjà.
mempool_empty_tick	Tick du validateur alors que la mempool est vide : aucun bloc n'est miné.
tick_ignored_waiting_db	Tick reçu pendant l'attente DB : il est ignoré par <code>waitingForDbState</code> .
invalid_signature_actor_branch	Branche signature invalide dans <code>MempoolActor</code> .

5 Vérification des propriétés structurelles

5.1 Définition des propriétés structurelles

Les propriétés structurelles sont des propriétés qui se déduisent de la forme du réseau : les places, les transitions et leurs arcs. Elles ne dépendent pas d'un calcul précis de hash ou de montant. Elles répondent à des questions comme :

- une transition peut-elle être tirée trop tôt ?
- un état d'attente peut-il rester bloqué ?
- deux comportements incompatibles peuvent-ils être actifs en même temps ?
- une transaction peut-elle disparaître sans confirmation ?

5.2 Absence de deadlock interne

On distingue un état terminal normal d'un vrai deadlock.

État terminal normal. Le système est stable : par exemple, la mempool est vide et le validateur est idle. Il n'y a rien à faire tant qu'aucune nouvelle requête externe ou aucun tick utile n'arrive.

Deadlock interne. Une place marquée comme `pending` contient un jeton, c'est-à-dire qu'une requête interne est en attente, mais aucune transition interne ne peut progresser. C'est le cas problématique recherché.

Formellement, pour un marquage M , on considère un deadlock interne si :

$$\exists p \in P_{\text{pending}}, M(p) > 0 \quad \text{et} \quad \nexists t \in T_{\text{interne}}, t \text{ activable dans } M.$$

Dans les scénarios représentatifs, aucun deadlock interne n'est attendu :

- une demande de solde peut progresser par `T_DB_BALANCE_RESPONSE` ;
- une demande de dernier bloc peut progresser par `T_DB_GET_LAST_BLOCK_RESPONSE` ;
- un bloc miné peut progresser par `T_DB_SAVE_BLOCK_REQUEST` ;
- une écriture DB en cours peut progresser soit vers `DB.Success`, soit vers `DB.Failed` ;
- un nettoyage mempool peut progresser vers un retrait total ou partiel.

La vérification bornée de l'onglet *Analyse* explore automatiquement les marquages atteignables et signale les deadlocks internes. Les bornes par défaut sont : profondeur maximale, nombre maximal d'états, nombre maximal de tokens par place et nombre maximal de tokens total. Cette exploration n'est pas une preuve mathématique infinie, mais elle permet de détecter les blocages réalistes dans le modèle.

5.3 Non-suppression avant confirmation

La propriété la plus importante est : **la mempool ne doit supprimer une transaction qu’après confirmation DB.**

Dans le modèle :

- `T_MEMPOOL_REMOVE_CONFIRMED_ALL` et `T_MEMPOOL_REMOVE_CONFIRMED_PARTIAL` nécessitent la place `P_MEMPOOL_CLEANUP_REQ` ;
- `P_MEMPOOL_CLEANUP_REQ` est produite uniquement par `T_VALIDATOR_CONFIRM_SAVED` ;
- `T_VALIDATOR_CONFIRM_SAVED` nécessite `P_BLOCK_PERSISTED` ;
- `P_BLOCK_PERSISTED` est produite uniquement par `T_DB_SAVE_SUCCESS`.

Donc toute trace contenant `RemoveTxs` doit contenir auparavant `DB.Success`. La chaîne causale est :

$$\text{DB.Success} \Rightarrow \text{P_BLOCK_PERSISTED} \Rightarrow \text{ConfirmSaved} \Rightarrow \text{P_MEMPOOL_CLEANUP_REQ} \Rightarrow \text{RemoveTxs}.$$

Cette propriété est structurelle : même sans connaître les valeurs des transactions, l’ordre imposé par les arcs empêche le nettoyage prématuré.

5.4 Conservation de la mempool après `DB.Failed`

En cas d’échec DB, le modèle doit représenter fidèlement le code : les transactions ne sont pas supprimées.

Dans le modèle :

- `T_DB_SAVE_FAILED` produit `P_DB_SAVE_FAILED` et `P_VALIDATOR_REJECT_TXS` ;
- `T_VALIDATOR_SAVE_FAILED` consomme ces places, répond `false`, puis remet le validateur idle ;
- aucune de ces transitions ne produit `P_MEMPOOL_CLEANUP_REQ` ;
- `P_MEMPOOL_TX_PENDING` reste donc marqué.

La propriété vérifiée est donc :

$$\text{DB.Failed} \Rightarrow \text{pas de nettoyage immédiat de la mempool}.$$

C’est cohérent avec `ValidatorActor.scala`, où la branche `SaveFailed` ne contient pas d’appel à `mempool ! Mempool.RemoveTxs(...)`.

5.5 `GetTxs` est non destructif

La transition `T_MEMPOOL_GET_TXS_NONEMPTY` représente `Mempool.GetTxs`. Dans le code, `GetTxs` fait :

```
val (toSend, _) = txs.splitAt(2)
replyTo ! Mempool.Txs(toSend)
Behaviors.same
```

La mempool envoie donc les transactions au validateur, mais ne les retire pas. Le modèle respecte cette propriété : la transition consomme `P_MEMPOOL_TX_PENDING` et la reproduit en sortie. Ainsi, la mempool reste non vide jusqu’à `RemoveTxs`.

Cette propriété est essentielle : si `GetTxs` supprimait immédiatement les transactions, un échec DB pourrait perdre les transactions. Le modèle empêche ce comportement.

5.6 Exclusion entre minage et attente DB

Le validateur ne doit pas lancer un nouveau cycle de minage pendant qu'il attend la réponse de la DB.

Dans le code réel, après `DB.SaveBlock`, le validateur passe dans `waitingForDbState`. Dans cet état :

- `ConfirmSaved` est accepté ;
- `SaveFailed` est accepté ;
- tous les autres messages, y compris `StartMining`, sont ignorés.

Dans le modèle, cela correspond à la place `P_VALIDATOR_WAITING_DB`. La transition `T_VALIDATOR_START_MINING` nécessite `P_VALIDATOR_IDLE`. Or `T_DB_SAVE_BLOCK_REQUEST` produit `P_VALIDATOR_WAITING_DB` et ne produit pas `P_VALIDATOR_IDLE`. Donc `StartMining` n'est pas activable tant que la DB n'a pas répondu.

La transition `T_VALIDATOR_TICK_IGNORED` modélise explicitement le cas où un tick arrive malgré tout pendant l'attente : il ne change pas l'état du validateur.

5.7 Validité de l'entrée en mempool

Une transaction ne doit entrer dans la mempool que si sa signature est valide.

Dans le modèle :

- `T_MEMPOOL_VERIFY_OK_ADD_EMPTY` et `T_MEMPOOL_VERIFY_OK_ADD_NONEMPTY` produisent `P_MEMPOOL_TX_PENDING` ;
- ces transitions correspondent au cas `Crypto.verify = true` ;
- `T_MEMPOOL_VERIFY_INVALID` produit `P_SIGNATURE_INVALID` et l'événement `txRejected`, mais ne produit pas `P_MEMPOOL_TX_PENDING`.

La structure du réseau garantit donc qu'une signature invalide ne crée pas une transaction en attente dans la mempool.

5.8 Refus local en cas de fonds insuffisants

Si `currentBalance < amount + fee`, le wallet doit refuser la transaction avant de la signer. Le modèle représente cette branche avec `T_WALLET_FUNDS_INSUFFICIENT`.

Cette transition produit :

- `P_TX_REJECTED_LOCAL` ;
- l'événement `insufficientFunds` ;
- l'événement `txRejected` ;
- le retour vers `P_WALLET_IDLE`.

Elle ne produit ni `P_UNSIGNED_TX`, ni `P_SIGNED_TX_SENT`, ni `P_MEMPOOL_ADD_REQ`. Le réseau garantit donc que les fonds insuffisants arrêtent le flux avant la mempool.

6 Invariants

6.1 Invariants structurels de places

Un invariant de places, ou P-invariant, exprime qu'une certaine combinaison de jetons reste constante sur tous les marquages atteignables. Dans ce projet, les invariants sont utilisés comme arguments de sûreté.

Invariant I1 – disponibilité de la DB

La place `P_DB_LEDGER` représente la présence de l'état interne de la DB : `lastHash`, `currentId`, `blocks` et le fichier `ledger.txt`. Les transitions qui consultent ou modifient la DB consomment ce jeton puis le reproduisent.

On a donc l'invariant :

$$M(P_DB_LEDGER) = 1.$$

Interprétation : la DB peut changer de contenu, mais elle ne disparaît pas du système. Après `DB.Success`, elle revient avec un état mis à jour ; après `DB.Failed`, elle revient avec son ancien état.

Invariant I2 – état principal de la mempool

En dehors des places historiques ou transitoires, la mempool est soit vide, soit non vide :

$$M(P_MEMPOOL_EMPTY) + M(P_MEMPOOL_TX_PENDING) = 1.$$

Interprétation : le modèle ne permet pas que la mempool soit simultanément vide et non vide dans son état principal. Les transitions d'ajout, de lecture et de nettoyage conservent cette cohérence.

Invariant I3 – GetTxs ne consomme pas la transaction

Pour la transition `T_MEMPOOL_GET_TXS_NONEMPTY`, la place `P_MEMPOOL_TX_PENDING` est à la fois entrée et sortie. Cela donne l'invariant local :

$$\Delta M(P_MEMPOOL_TX_PENDING) = 0 \quad \text{lors de } \text{GetTxs}.$$

Interprétation : le validateur peut sélectionner un lot, mais les transactions restent dans la mempool jusqu'à la confirmation DB.

Invariant I4 – synchronisation SaveBlock / waiting DB

Après `T_DB_SAVE_BLOCK_REQUEST`, deux places sont produites ensemble :

- `P_DB_SAVE_IN_PROGRESS` ;
- `P_VALIDATOR_WAITING_DB`.

Les réponses `T_DB_SAVE_SUCCESS` et `T_DB_SAVE_FAILED` consomment ces deux places ensemble. On obtient donc l'invariant de synchronisation suivant sur les états atteignables du cycle de persistance :

$$M(P_DB_SAVE_IN_PROGRESS) = M(P_VALIDATOR_WAITING_DB).$$

Interprétation : si la DB est en train de sauvegarder un bloc, alors le validateur est nécessairement en attente de cette réponse. Réciproquement, si le validateur attend la DB, une sauvegarde est en cours.

Invariant I5 – pas de nettoyage sans demande explicite

Les transitions de nettoyage de la mempool exigent `P_MEMPOOL_CLEANUP_REQ`. Cette place n'est produite que par `T_VALIDATOR_CONFIRM_SAVED`. Donc :

$$\text{mempoolCleaned} \Rightarrow \text{removeRequested} \Rightarrow \text{txAccepted}.$$

Interprétation : le nettoyage n'est pas autonome. La mempool ne décide pas seule de supprimer des transactions ; elle attend une demande explicite du validateur après confirmation DB.

6.2 Invariants métier

Les invariants métier traduisent les règles attendues de la blockchain simulée.

Invariant métier	Justification dans le modèle	Limite
Une transaction insuffisamment financée n'entre pas en mempool.	La branche <code>T_WALLET_FUNDS_INSUFFICIENT</code> produit <code>txRejected</code> et ne produit pas <code>P_SIGNED_TX_SENT</code> .	Le modèle ne recalcule pas les montants ; il suppose la branche de test fournie par le code.
Une signature invalide est rejetée.	<code>T_MEMPOOL_VERIFY_INVALID</code> produit <code>signatureInvalid</code> et <code>txRejected</code> , sans produire <code>P_MEMPOOL_TX_PENDING</code> .	Le modèle ne vérifie pas RSA ; il abstrait <code>Crypto.verify</code> .
Une transaction sélectionnée par le validateur reste en mempool tant que la DB n'a pas confirmé.	<code>GetTxs</code> est non destructif ; <code>RemoveTxs</code> nécessite <code>DB.Success</code> .	La cardinalité exacte de la mempool est abstraite.
Un bloc confirmé correspond à un bloc miné.	<code>T_DB_SAVE_BLOCK_REQUEST</code> nécessite <code>P_BLOCK_MINED</code> .	Le hash et le nonce exacts ne sont pas recalculés.
Un échec DB ne nettoie pas la mempool.	La branche <code>DB.Failed</code> ne produit pas <code>P_MEMPOOL_CLEANUP_REQ</code> .	Ce choix peut créer un retry implicite après une réponse API négative.
Le validateur ne mine pas deux blocs simultanément dans le même cycle.	<code>waitingForDbState</code> empêche <code>StartMining</code> d'être activable comme nouveau cycle utile.	Le modèle suppose la discipline d'acteur Akka : un message traité à la fois par acteur.

6.3 Point d'attention sur le solde

Le réseau de Pétri ne prouve pas directement l'invariant numérique : *un solde ne devient jamais négatif*. Il vérifie plutôt le contrôle associé : si le solde observé est insuffisant, la transaction est rejetée.

Dans le code réel, le `WalletActor` décrémente localement `initialBalance` dès l'envoi à la mempool, avant confirmation DB :

```
val newState = state.copy(  
  initialBalance = state.initialBalance - amount - fee,  
  nonce = state.nonce + 1  
)
```

Cette stratégie permet d'éviter que deux transactions en file utilisent le même solde. En revanche, en cas d'échec DB, il n'y a pas de rollback explicite du solde local. Le modèle signale cette limite comme un écart entre vérification du flux de contrôle et vérification des valeurs métier.

7 Propriétés LTL

7.1 Rappel simple sur LTL

La LTL, ou *Linear Temporal Logic*, sert à exprimer des propriétés sur des traces d'exécution. Une trace est une suite d'événements observés pendant l'exécution du système.

Les opérateurs utilisés ici sont :

- $G\varphi$: *globally*, la propriété φ doit être vraie à toutes les étapes ;
- $F\varphi$: *finally*, la propriété φ doit finir par devenir vraie ;
- $X\varphi$: *next*, la propriété φ doit être vraie à l'étape suivante ;
- $\varphi \rightarrow \psi$: si φ est vraie, alors ψ doit l'être selon l'opérateur temporel donné.

Exemple :

$$G(\text{txCreated} \rightarrow F(\text{txAccepted} \vee \text{txRejected}))$$

se lit : à chaque fois qu'une transaction est créée, elle doit finir par être acceptée ou rejetée.

7.2 Propositions atomiques

Les propositions atomiques sont attachées aux transitions du réseau. Elles ne sont pas des états complets, mais des événements observés.

Proposition atomique	Signification
<code>txCreated</code>	Le wallet commence le traitement d'une transaction.
<code>txAccepted</code>	Le validateur confirme une transaction après <code>DB.Success</code> .
<code>txRejected</code>	Une transaction est refusée localement, par la mempool ou après <code>DB.Failed</code> .
<code>blockMined</code>	Le proof of work abstrait a produit un bloc.
<code>dbSaved</code>	La DB a répondu <code>Success</code> .
<code>dbFailed</code>	La DB a répondu <code>Failed</code> .
<code>mempoolCleaned</code>	La mempool a supprimé des transactions confirmées.
<code>insufficientFunds</code>	Le wallet a détecté un solde insuffisant.
<code>signatureInvalid</code>	La mempool a rejeté la signature.
<code>validatorWaitingDb</code>	Le validateur est passé en attente de réponse DB.
<code>startMining</code>	Un tick de minage lance une interrogation de la mempool.
<code>tickIgnored</code>	Un tick reçu pendant <code>waitingForDbState</code> est ignoré.

7.3 Formules vérifiées

Id	Formule LTL	Lecture simple
L1	$G(\text{txCreated} \rightarrow F(\text{txAccepted} \vee \text{txRejected}))$	Toute transaction créée doit finir acceptée ou rejetée.
L2	$G(\text{blockMined} \rightarrow F(\text{dbSaved} \vee \text{dbFailed}))$	Tout bloc miné doit recevoir une réponse DB.
L3	$G(\text{dbSaved} \rightarrow F(\text{mempoolCleaned}))$	Après une persistance réussie, la mempool doit être nettoyée.
L4	$G(\text{dbFailed} \rightarrow F(\text{txRejected}))$	Après un échec DB, les transactions sélectionnées doivent recevoir un refus.
L5	$G(\text{insufficientFunds} \rightarrow F(\text{txRejected}))$	Des fonds insuffisants doivent produire un rejet.
L6	$G(\text{signatureInvalid} \rightarrow F(\text{txRejected}))$	Une signature invalide doit produire un rejet.
L7	$G(\text{dbFailed} \rightarrow X(\neg \text{mempoolCleaned}))$	Juste après un échec DB, on ne doit pas nettoyer la mempool.

Id	Formule LTL	Lecture simple
L8	$G(\text{validatorWaitingDb} \rightarrow X(\neg \text{startMining} \vee \text{tickIgnored}))$	Quand le validateur attend la DB, le prochain tick ne doit pas lancer un nouveau minage utile.

7.4 Résultat sur les traces de scénarios

Sur les scénarios témoins, les formules LTL servent à vérifier que la trace jouée respecte le comportement attendu.

- Sur `transfer_success`, L1, L2 et L3 sont satisfaites : la transaction est créée, minée, persistée, confirmée, puis supprimée de la mempool.
- Sur `db_failure_retry_possible`, L2, L4 et L7 sont satisfaites : le bloc miné reçoit une réponse DB négative, les transactions reçoivent un refus, et la mempool n'est pas nettoyée immédiatement.
- Sur `insufficient_funds`, L5 est satisfaite : l'insuffisance de fonds mène directement à `txRejected`.
- Sur `invalid_signature_actor_branch`, L6 est satisfaite : la signature invalide mène directement à `txRejected`.
- Sur `tick_ignored_waiting_db`, L8 est satisfaite : le tick reçu pendant l'attente DB est modélisé par `tickIgnored` et ne lance pas un nouveau cycle de minage.

Le panneau LTL du visualiseur évalue les formules sur le préfixe de trace courant. Si le scénario n'est pas encore terminé, une propriété peut apparaître *en attente*. Cela signifie qu'un déclencheur a été observé, mais que la conséquence attendue n'a pas encore été jouée dans le préfixe courant.

7.5 Résultat sur l'espace d'états borné

L'onglet *Analyse* explore les marquages atteignables par BFS dans des bornes configurables. Cette vérification est plus forte qu'un scénario unique, mais elle reste bornée.

La configuration prévue pour analyser un cycle transactionnel représentatif est :

- une requête externe représentative ;
- profondeur maximale bornée ;
- nombre d'états maximal borné ;
- nombre de tokens par place borné ;
- heuristique de fairness évitant les boucles temporelles sans intérêt ;
- mode *1 requête* pour neutraliser la concurrence artificielle de `pendingTx`s pendant l'analyse du cycle principal.

Sous ces hypothèses, l'analyse sert à confirmer :

- absence de deadlock interne dans les marquages explorés ;
- absence de nettoyage de la mempool sans `DB.Success` ;
- absence de nettoyage immédiat après `DB.Failed` ;
- impossibilité de lancer un minage utile pendant `waitingForDbState` ;
- satisfaction des propriétés de vivacité sur les chemins représentatifs.

7.6 Pourquoi il faut des hypothèses de justice

Les propriétés L1 à L4 sont des propriétés de vivacité : elles disent qu'une conséquence doit finir par arriver. Or, dans un réseau de Pétri purement non déterministe, si on autorise toutes les transitions

à tout moment, on peut construire des traces peu réalistes qui retardent indéfiniment la transition attendue.

Exemple : après `txCreated`, une exploration totalement non contrainte peut continuer à tirer des transitions externes indépendantes ou des boucles qui ne font pas avancer la transaction courante. La formule L1 peut alors paraître violée, non pas parce que le code ne sait pas confirmer ou rejeter la transaction, mais parce que la sémantique explorée n'impose pas de justice.

Il faut donc distinguer :

- **sûreté** : quelque chose de mauvais ne doit jamais arriver. Exemple : `RemoveTx` avant `DB.Success`. Ces propriétés sont très bien capturées structurellement.
- **vivacité** : quelque chose de bon doit finir par arriver. Exemple : une transaction créée finit acceptée ou rejetée. Ces propriétés nécessitent des hypothèses de progression : la DB finit par répondre, le mineur finit son calcul, le scheduler Akka traite les messages, et l'utilisateur n'injecte pas une infinité d'actions qui empêchent le scénario courant d'avancer.

Cette distinction est normale en vérification formelle. Le rapport considère donc les propriétés de sûreté comme les plus fortes, et les propriétés de vivacité comme valides sous hypothèse de fairness raisonnable du système.

8 Comparaison entre comportement réel et modèle formel

8.1 Correspondance étape par étape

Comportement réel Akka	Transition Petri	Commentaire
La route HTTP résout le wallet via <code>Registry.GetWalletRef</code> .	<code>T_REGISTRY_GET_WALLET_FOUND</code> ou <code>T_REGISTRY_GET_WALLET_NOT_FOUND</code>	Le modèle garde seulement le résultat : trouvé ou absent.
Le wallet reçoit <code>CreateTx</code> .	<code>T_WALLET_CREATE_TX_START</code>	Correspond au passage <code>txInProgress = true</code> .
Le wallet demande le solde courant.	<code>T_WALLET_GET_BALANCE_REQUEST</code> , puis <code>T_DB_BALANCE_RESPONSE</code>	Le calcul du solde dans le ledger est abstrait par une réponse.
Le wallet construit une transaction non signée.	<code>T_WALLET_FUNDS_OK_BUILD_UNSIGNED</code>	La branche opposée est <code>T_WALLET_FUNDS_INSUFFICIENT</code> .
Le wallet signe et envoie à la mempool.	<code>T_WALLET_SIGN_SEND_MEMPOOL</code> , puis <code>T_MEMPOOL_ADD_TX</code>	Le hash et RSA sont abstraits par des événements.
La mempool vérifie la signature.	<code>T_MEMPOOL_VERIFY_OK_ADD_EMPTY</code> , <code>T_MEMPOOL_VERIFY_OK_ADD_NONEMPTY</code> ou <code>T_MEMPOOL_VERIFY_INVALID</code>	Le modèle distingue signature valide et invalide.
Le timer du validateur déclenche <code>StartMining</code> .	<code>T_VALIDATOR_START_MINING</code>	Le délai réel de 5 secondes est abstrait.
La mempool renvoie les transactions sans les supprimer.	<code>T_MEMPOOL_GET_TXS_NONEMPTY</code>	Propriété essentielle : <code>GetTx</code> est non destructif.
Le validateur demande le dernier bloc.	<code>T_DB_GET_LAST_BLOCK_REQUEST</code> , puis <code>T_DB_GET_LAST_BLOCK_RESPONSE</code>	Le hash précédent et l'id courant sont abstraits.
Le validateur mine le bloc.	<code>T_MINER_START</code> , <code>T_MINER_PROOF_OF_WORK</code>	Le calcul potentiellement long devient une transition.

Comportement réel Akka	Transition Petri	Commentaire
Le validateur demande la sauvegarde.	T_DB_SAVE_BLOCK_REQUEST	Le validateur passe dans <code>waitingForDbState</code> .
La DB répond <code>Success</code> .	T_DB_SAVE_SUCCESS, puis T_VALIDATOR_CONFIRM_SAVED	Les wallets reçoivent <code>true</code> , puis la mempool est nettoyée.
La DB répond <code>Failed</code> .	T_DB_SAVE_FAILED, puis T_VALIDATOR_SAVE_FAILED	Les wallets reçoivent <code>false</code> , mais la mempool n'est pas nettoyée.

8.2 Ce que le modèle reproduit fidèlement

Le modèle reproduit fidèlement les éléments suivants :

- l'ordre des messages principaux entre acteurs ;
- l'attente asynchrone du wallet pour le solde ;
- le fait que `GetTx`s ne vide pas la mempool ;
- le fait que `RemoveTx`s arrive seulement après `DB.Success` ;
- le fait que `DB.Failed` ne déclenche pas `RemoveTx`s ;
- le blocage volontaire du validateur en `waitingForDbState` ;
- les branches de rejet : wallet absent, fonds insuffisants, signature invalide, DB failed.

8.3 Différences entre le modèle et le code réel

Aspect	Code réel	Modèle Petri
Temps	Le validateur reçoit un tick toutes les 5 secondes. Le minage peut prendre un temps variable.	Le temps est abstrait : <code>StartMining</code> et <code>mine()</code> sont des transitions.
Cryptographie	Le code calcule un hash SHA-256 et une signature RSA simplifiée.	Le modèle garde seulement <code>signatureValid</code> ou <code>signatureInvalid</code> .
Mempool	La liste contient un nombre exact de transactions triées par fee.	Le modèle distingue surtout <i>vide</i> , <i>non vide</i> , et <i>reste après cleanup</i> .
Solde	Le wallet combine solde local et solde issu du ledger.	Le modèle ne manipule pas de valeurs numériques.
DB	Le code écrit réellement dans <code>ledger.txt</code> .	La réussite ou l'échec DB est une branche non déterministe.
Concurrence Akka	Chaque acteur traite ses messages séquentiellement, mais plusieurs acteurs s'exécutent de façon concurrente.	Le réseau représente les interleavings possibles par non-déterminisme.
Réponse API	<code>ask[Boolean]</code> attend une réponse du wallet/validateur.	La réponse est représentée par <code>txAccepted</code> ou <code>txRejected</code> .

8.4 Écart métier principal : échec DB et retry implicite

Le modèle met en évidence un comportement important du code réel : après `DB.Failed`, le validateur répond `false` aux wallets, mais les transactions restent en mempool. Cela signifie qu'un tick ultérieur peut les reprendre et potentiellement les miner plus tard.

Deux interprétations sont possibles :

1. **Retry implicite voulu** : l'échec DB est considéré comme temporaire, donc la transaction reste en attente pour une prochaine tentative. Dans ce cas, il serait plus cohérent de ne pas répondre définitivement `false` à l'API, ou d'indiquer une réponse du type *pending/retry*.
2. **Rejet définitif voulu** : l'échec DB annule la tentative. Dans ce cas, il faudrait aussi retirer la transaction de la mempool ou la marquer comme rejetée pour éviter qu'elle soit minée plus tard.

Le réseau de Pétri ne choisit pas à la place du code ; il rend simplement ce comportement visible. Pour un rapport de vérification, c'est un point intéressant, car il montre que le modèle ne sert pas seulement à valider le système : il aide aussi à repérer les ambiguïtés métier.

9 Résultats synthétiques

Propriété	Statut	Justification
Absence de nettoyage avant confirmation DB	OK	<code>RemoveTx</code> s nécessite <code>P_MEMPOOL_CLEANUP_REQ</code> , produit uniquement après <code>DB.Success</code> .
Conservation de la mempool après <code>DB.Failed</code>	OK	La branche <code>SaveFailed</code> ne produit pas de demande de cleanup.
<code>GetTx</code> s non destructif	OK	La place <code>P_MEMPOOL_TX_PENDING</code> est reproduite après sélection du batch.
Rejet des fonds insuffisants	OK	La branche <code>T_WALLET_FUNDS_INSUFFICIENT</code> rejette avant signature et avant mempool.
Rejet des signatures invalides	OK	La branche <code>T_MEMPOOL_VERIFY_INVALID</code> ne produit pas de transaction pending.
Pas de nouveau minage utile pendant <code>waitingForDbState</code>	OK	<code>StartMining</code> nécessite <code>P_VALIDATOR_IDLE</code> ; l'attente DB utilise <code>P_VALIDATOR_WAITING_DB</code> .
Absence de deadlock interne dans les scénarios représentatifs	OK	Chaque place pending possède une transition de réponse ou de progression dans les chemins modélisés.
Vivacité globale L1–L4 sans hypothèse de fairness	À discuter	Une exploration non contrainte peut retarder indéfiniment certaines conséquences ; il faut borner l'analyse et supposer que les réponses DB/minage/scheduler finissent par progresser.
Invariant numérique de solde non négatif	À discuter	Le modèle contrôle la branche <i>fonds insuffisants</i> , mais ne prouve pas les calculs exacts de solde.
Politique après <code>DB.Failed</code>	À discuter	Le code répond <code>false</code> mais conserve la transaction en mempool, ce qui crée un retry implicite.

10 Limites de la vérification

Les limites principales sont les suivantes :

- **Exploration bornée** : l’onglet Analyse limite la profondeur, le nombre d’états et le nombre de tokens. Ce n’est pas une preuve exhaustive sur un système infini.
- **Abstraction des valeurs** : les montants, fees, hashes, nonces et signatures ne sont pas recalculés dans le modèle.
- **Cardinalité simplifiée de la mempool** : le modèle distingue surtout vide / non vide / reste, au lieu de compter exactement toutes les transactions.
- **Fairness nécessaire** : les propriétés de vivacité supposent que les messages utiles finissent par être traités et que la DB finit par répondre.
- **Sémantique Akka simplifiée** : chaque acteur traite ses messages séquentiellement, mais le modèle ne reproduit pas tous les détails du scheduler Akka.
- **Réponse API après échec DB** : le modèle révèle un choix métier qui peut être discutable, mais ne le corrige pas automatiquement.

Ces limites ne rendent pas la vérification inutile. Au contraire, elles clarifient précisément ce qui est prouvé par le modèle et ce qui reste hors périmètre.

11 Conclusion

Le réseau de Pétri fournit une représentation claire et vérifiable du comportement critique de la blockchain Akka/Scala. Il montre que le système respecte les propriétés de sûreté essentielles : les transactions valides ne sont supprimées qu’après confirmation DB, les échecs DB ne nettoient pas la mempool, les signatures invalides et les fonds insuffisants sont rejetés, et le validateur ne lance pas un nouveau cycle utile pendant l’attente de persistance.

Les invariants structurels expliquent pourquoi ces propriétés tiennent : certains jetons sont conservés, certaines transitions ne peuvent être tirées qu’après des événements précis, et l’état `waitingForDbState` synchronise correctement le validateur avec la DB.

La LTL complète cette analyse en exprimant des propriétés temporelles faciles à lire : toute transaction créée doit finir acceptée ou rejetée, tout bloc miné doit recevoir une réponse DB, et une persistance réussie doit mener au nettoyage de la mempool. Ces propriétés sont validées sur les scénarios témoins et sur l’exploration bornée sous hypothèses réalistes de fairness.

Enfin, la comparaison avec le code réel montre que le modèle n’est pas seulement décoratif : il correspond aux messages Akka et met en évidence un point métier important après `DB.Failed`. Le projet dispose donc d’une base formelle utile pour expliquer, tester et améliorer le comportement de la blockchain simulée.

Références

- [1] T. Murata, *Petri Nets : Properties, Analysis and Applications*, Proceedings of the IEEE, 1989.
- [2] W. Reisig, *Understanding Petri Nets*, Springer, 2013.
- [3] C. Baier, J.-P. Katoen, *Principles of Model Checking*, MIT Press, 2008.
- [4] L. Lamport, *The Temporal Logic of Actions*, ACM TOPLAS, 1994.
- [5] Akka Documentation, *Akka Typed Actors*, <https://doc.akka.io/>.